

Security Audit Report

Date: March 8, 2026

Scope: Full codebase — auth, API, tools, channels, database, secrets

Severity	Count	Examples
CRITICAL	5	Leaked API key, command injection, insecure RNG
HIGH	10	Master key bypass, missing rate limits, SSRF
MEDIUM	12	Missing CSP, setup re-run, default credentials
LOW	8	Timing leak, verbose logging, password policy

1. Critical Findings

[CRITICAL] Exposed API Key in Git History

.mcp.json — commit 92e6f13

A real `ASSISTANT_API_KEY` was committed in plaintext. Even though the file may have been modified since, the key is permanently visible in git history to anyone who clones the repo.

Fix: Revoke the key immediately. Use BFG Repo-Cleaner or git filter-repo to purge from history. Add `.mcp.json` to `.gitignore`.

[CRITICAL] Command Injection in Docker Tool

src/tools/docker/index.ts:117-188

All Docker operations (start, stop, logs, build, exec) concatenate user input directly into shell commands via string interpolation: `execAsync(`docker ${cmd}`)`. A container name like `foo; rm -rf /` achieves arbitrary code execution.

Fix: Replace with `spawn('docker', [...argsArray])` using array-based arguments, matching the safe pattern already used by git/github tools.

[CRITICAL] Command Injection in 'which' Tool

src/tools/shell/index.ts:143

The `which` tool passes user input directly into a shell command via string interpolation and has `requiresPermission: false`. Input like `foo; cat /etc/passwd` executes the injected command with no approval.

Fix: Use `execFile('which', [args.name])` or validate input to alphanumeric/hyphens only.

[CRITICAL] Insecure RNG for TOTP Backup Codes

src/security/auth/totp.ts:58

`Math.random()` is used to generate TOTP backup codes. These serve as authentication bypass tokens and must be cryptographically unpredictable.

Fix: Replace with `crypto.randomBytes()` or `crypto.getRandomValues()`.

[CRITICAL] Insecure RNG for Channel Linking Codes

src/channels/linking.ts:80

`Math.random()` generates 6-character linking codes that bind channel identities (Telegram, Slack, Teams) to user accounts. The small keyspace combined with predictable PRNG makes brute-force feasible.

Fix: Use `crypto.randomBytes()` for code generation.

2. High Severity Findings

[HIGH] MASTER_KEY Grants Unrestricted Admin Access

src/api/server.ts:114-121

The MASTER_KEY environment variable can be used as a Bearer token to authenticate as a full admin with no audit trail (session is null). Any service or leak of this key gives complete API control.

Fix: Restrict MASTER_KEY to specific internal endpoints only, or replace with proper service accounts. Add audit logging when system user authenticates.

[HIGH] No Rate Limiting on Register / Passkey / Link Endpoints

src/api/routes/auth.ts

Rate limiting and account lockout are only applied to /login. The /register, /passkey/auth/verify, and /auth/link endpoints have no rate limits, enabling brute-force and credential stuffing.

Fix: Apply `rateLimiter.check()` and `recordFailedLogin()` to all auth endpoints.

[HIGH] Python Code Injection in Voice Engines

src/voice/wake-word.ts, src/voice/stt.ts

Model paths and access keys are interpolated directly into Python script strings executed via `python3 -c`. Malicious values can break out of the string context for arbitrary code execution.

Fix: Pass parameters via a JSON temp file or environment variables instead of string interpolation.

[HIGH] Vault Master Key Has No KDF Stretching

src/security/vault.ts:24

The vault master key is derived via a single SHA-256 hash with no salt and no iterations. Low-entropy master keys are trivially brute-forceable.

Fix: Use `scryptSync` or `Argon2id` with a fixed salt for deterministic derivation with proper stretching.

[HIGH] SSRF via Outbound Webhook URLs

src/hooks/actions.ts:120-142

The `executeWebhook` action fetches URLs from user-configurable hook config stored in the database. No validation that the URL points to an external host allows requests to internal services.

Fix: Validate webhook URLs against a denylist of private/internal IP ranges using the existing `validateExternalUrl()`.

[HIGH] XSS Risk in OAuth Error Page + Unsafe `postMessage`

src/api/routes/oauth.ts:155-176

Error messages are interpolated into JavaScript with only single-quote escaping. Additionally, `postMessage` uses `origin '*'`, broadcasting OAuth callbacks to any parent window.

Fix: Use `JSON.stringify()` for JS interpolation. Specify target origin instead of `'*'`.

[HIGH] Webhook Signature Verification is Optional

src/api/routes/webhooks.ts:80-86

Hooks without configured secrets allow unauthenticated trigger by anyone who discovers the webhook path. The code warns but still processes unsigned payloads.

Fix: Make webhook secrets mandatory. Reject unsigned webhooks with 401.

[HIGH] Missing Redis Authentication Enforcement

src/db/redis.ts

Redis client initialization doesn't enforce authentication. If deployed without a password, any network-adjacent attacker can read/write cached sessions and secrets.

Fix: Require REDIS_PASSWORD in production configurations. Fail startup if not set in external storage mode.

[HIGH] SQL Injection in Migration Code

src/db/migrate.ts:109

String interpolation in SQL INSERT for migration tracking: `VALUES( '${hash}', ${entry.when} )`. While hash is computed internally, the pattern is dangerous.

Fix: Use parameterized queries for all SQL, including migrations.

[HIGH] Hardcoded Database Credential Fallbacks

src/config/defaults.ts:6, src/config/schema.ts:8

Fallback connection string `postgres://assistant:assistant@localhost:5432/assistant` means production could silently use known default credentials if DATABASE_URL is missing.

Fix: Remove fallbacks. Throw an explicit error if DATABASE_URL is not configured.

3. Medium Severity Findings

[MEDIUM] No Content-Security-Policy Header

src/api/server.ts:66-71

Security headers like X-Frame-Options are set, but CSP and HSTS are missing. This leaves the web UI more exposed to XSS attacks.

[MEDIUM] Setup Wizard Can Be Re-Run

src/api/routes/settings.ts:288-312

setupComplete is just a database flag with no enforcement at boot. If cleared, the setup wizard can be re-run and a new admin registered.

[MEDIUM] MCP Server HTTP Transport Has No Auth

mcp-server/src/index.ts:47-49

In HTTP mode, the MCP server sets Access-Control-Allow-Origin: * with zero authentication on /sse and /messages endpoints.

[MEDIUM] Workspace API Accepts Arbitrary Paths

src/api/routes/workspace.ts:21-43

PUT /api/workspace allows adding arbitrary directories (including /, /etc) to additionalPaths, bypassing filesystem tool path validation.

[MEDIUM] /tmp Unconditionally Allowed

src/tools/filesystem/index.ts:318

Any path under /tmp/assistant- is always allowed. Shared temp directories can be used to read/write files from other processes.

[MEDIUM] Shell Tool Blocklist is Bypassable

src/tools/shell/index.ts:10-48

Command validation uses a blocklist approach. Obfuscated commands, hex encoding, base64, or variable expansion can bypass it.

[MEDIUM] Session Tokens Returned in Response Body

src/api/routes/auth.ts:74-89

Session tokens are returned as JSON in addition to HttpOnly cookies. This allows storage in localStorage, which is vulnerable to XSS theft.

[MEDIUM] Default Telegram Authorization is Allow-All

src/channels/telegram/index.ts:128-133

If allowedUsers is empty (default), all Telegram users who know the bot username can send commands.

[MEDIUM] Health Endpoint Exposes Infrastructure Details

src/api/routes/health.ts:15-68

/api/health/detailed is public and exposes system uptime, agent count, database/Redis status with error messages.

[MEDIUM] Hardcoded LiteLLM API Key Fallback

src/api/routes/models.ts:189

Fallback key 'sk-litellm-master-key' is used when no key is configured. This is a predictable, well-known default.

[MEDIUM] PII Filter Stores Original Values

src/core/orchestrator/pii-filter.ts:69-74

The filterPII function returns redaction records containing the original PII value. If logged or returned to clients, it defeats PII filtering.

[MEDIUM] Error Messages Leak Internal Details

src/api/routes/agents.ts, models.ts, health.ts

Raw error messages including stack traces and internal service names are returned to API clients.

4. Low Severity Findings

[LOW] secureCompare Leaks Length Info

src/utis/crypto.ts:143-154

Early return on length mismatch partially defeats constant-time comparison.

[LOW] Env Tool Keyword-Based Redaction is Bypassable

src/tools/shell/index.ts:150-170

Sensitive env var redaction uses a keyword denylist that misses non-standard names.

[LOW] WebSocket Token in URL Query Parameter

src/api/websocket.ts:20-26

Tokens in query strings can appear in server logs. Standard for WS but should be documented.

[LOW] No Strict-Transport-Security Header

src/api/server.ts

HSTS header is missing. Browsers won't enforce HTTPS on subsequent visits.

[LOW] Password Policy is Minimal

src/api/routes/auth.ts:199

Only minLength: 8 enforced. No complexity rules or breached-password checking.

[LOW] Passkey Challenges Stored In-Memory Only

src/security/auth/passkey.ts:21

WebAuthn challenges in a Map break multi-instance deployments. Should use Redis.

[LOW] OAuth Token Exchange Error Leaks Provider Response

src/security/oauth.ts

Error details from OAuth providers may expose internal endpoint info.

[LOW] Default HOST Binds to 0.0.0.0

src/config/defaults.ts

Server binds to all interfaces by default, exposing it to the network.

5. Security Strengths

- AES-256-GCM encryption with random IVs and proper auth tag handling
- Argon2id password hashing via Bun.password with strong parameters (64MB, 3 iterations)
- Session tokens hashed (SHA-256) before Redis storage — no plaintext in cache
- OAuth with PKCE (S256) and single-use server-side state with TTL
- Cookie security: HttpOnly, Secure, SameSite=Strict on all auth cookies
- Credential enumeration protection: generic error messages on login failure
- Account lockout with escalating durations (15 min / 30 min / 60 min)
- Passkey implementation with counter verification and origin/RP ID checks
- Drizzle ORM with parameterized queries for all normal database operations
- Git/GitHub/GitLab tools use spawn() with array arguments (no shell injection)
- No eval() usage found anywhere in the codebase
- .env files properly excluded via .gitignore
- SSRF protection via validateExternalUrl() on browser and websearch tools
- Symlink resolution (realpathSync) in filesystem path validation
- Safe regex validation with complexity limits to prevent ReDoS
- Structured pino logging instead of console.log

6. Remediation Roadmap

Immediate — before any deployment

- Revoke the exposed ASSISTANT_API_KEY and purge from git history
- Fix command injection in Docker tool and 'which' tool (use spawn/execFile)
- Replace Math.random() with crypto.randomBytes() in TOTP and linking codes
- Restrict or remove MASTER_KEY bearer token bypass; add audit logging
- Remove hardcoded database credential fallbacks; fail on missing config

Short-term — within 2 weeks

- Apply rate limiting to all auth endpoints (register, passkey, link)
- Fix Python code injection in voice engines (use JSON file for params)
- Add KDF stretching to vault master key derivation
- Validate webhook URLs against private IP ranges
- Make webhook signature verification mandatory
- Fix OAuth error page XSS and postMessage origin
- Enforce Redis authentication in production config

Medium-term — within 1-2 months

- Add Content-Security-Policy and HSTS headers
- Replace shell command blocklist with allowlist or sandboxing
- Add authentication to MCP server HTTP transport
- Restrict workspace API from adding system directories
- Move passkey challenges from in-memory Map to Redis
- Stop returning session tokens in JSON response body
- Lock down setup wizard after initial configuration
- Sanitize error messages returned to API clients